ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

«Алгоритмы и структуры данных»

отчет по блоку задач №3 (Яндекс.Контест)

Выполнил:

Студент группы P3213

Якименко Владислав Игоревич

Преподаватель:

Сорокин Артём Николаевич

Санкт-Петербург, 2026г.

Задачи из 3-го блока Яндекс.Контеста

Задача «I. Машинки»

Дан порядок, в котором Петя хочет играть машинками. На полу одновременно может лежать не более K машинок. Нужно определить минимальное число обращений к маме, если мама заранее знает всю последовательность и каждый раз может убрать на полку самую выгодную машинку.

Формат ввода

Во входных данных задаются N, K и P — количество различных машинок, размер доступного места на полу и длина последовательности запросов. Затем идут P номеров машинок в порядке, в котором Петя захочет ими играть.

Формат вывода

Нужно вывести минимальное число операций снятия машинки с полки.

Идея решения:

Это задача на оптимальную замену в кэше. Если нужная машинка уже на полу, ничего делать не нужно.

Для каждой позиции я заранее считаю, когда эта же машинка понадобится в следующий раз. Это удобно делать проходом справа налево.

Среди машинок, которые сейчас лежат на полу, выгоднее всего убирать ту, которая понадобится позже всех, либо не понадобится больше вообще.

Поэтому на полу я храню машинки в множестве по их ближайшему следующему использованию. Когда места нет, убираю элемент с самым дальним следующим появлением.

Сложность:

Время: O(P log K + N)

Память: O(P + N)

Код:

```
/**
 * @author: Slava Yakimenko
 */


#include <iostream>
#include <set>
#include <vector>


using namespace std;
typedef long long ll;


#define fastIO()                \
```

```
  do {                            \
    ios::sync_with_stdio(false); \
    cin.tie(nullptr);             \
    cout.tie(nullptr);            \
  } while (0)
#define deb(x) cout << #x << " = " << x << '\n'
#define all(x) x.begin(), x.end()
#define el '\n'
#define INF static_cast<ll>(1e17)

/*
 * HEADER END
 */

void solve() {
  ll n = 0, k = 0, p = 0;
  cin >> n >> k >> p;

  vector<ll> a(p);
  for (ll i = 0; i < p; ++i) {
    cin >> a[i];
  }

  const ll no_next = p + 1;
  vector<ll> next_pos(p, no_next);
  vector<ll> last(n + 1, no_next);

  for (ll i = p - 1; i >= 0; --i) {
    next_pos[i] = last[a[i]];
    last[a[i]] = i;
  }

  ll ans = 0;
  vector<ll> current_next(n + 1, no_next);
  vector<bool> on_floor(n + 1, false);
  set<pair<ll, ll>> floor_cars;
```

```
  for (ll i = 0; i < p; ++i) {
    const ll car = a[i];

    if (on_floor[car]) {
      floor_cars.erase({current_next[car], car});
    } else {
      ++ans;
      if (static_cast<ll>(floor_cars.size()) == k) {
        auto it = prev(floor_cars.end());
        on_floor[it->second] = false;
        floor_cars.erase(it);
      }
      on_floor[car] = true;
    }

    current_next[car] = next_pos[i];
    floor_cars.insert({current_next[car], car});
  }

  cout << ans << el;
}

int main() {
  fastIO();
  // freopen("input.txt", "r", stdin);
  // freopen("output.txt", "w", stdout);
  bool multitest = false;
  ll T = 1;
  if (multitest) {
    cin >> T;
  }
  for (ll i = 0; i < T; ++i) {
    solve();
  }
}
```

Задача «J. Гоблины и очереди»

Нужно обработать очередь гоблинов с тремя типами запросов: обычный гоблин встает в конец, привилегированный — в середину, а запрос «-» удаляет первого гоблина и требует вывести его номер.

Формат ввода

Сначала задается число N — количество запросов. Далее идут N строк, каждая из которых содержит запрос вида + i, * i или -.

Формат вывода

Для каждого запроса типа «-» нужно вывести номер гоблина, который уходит к шаманам.

Идея решения:

Я храню очередь в двух деках: левая половина и правая половина.

Поддерживаю инвариант: в левой части либо столько же элементов, сколько в правой, либо на один больше. Тогда первый элемент всей очереди всегда находится в начале левого дека.

Запрос + добавляет элемент в конец правой части, запрос * — в начало правой части, а после этого вызывается балансировка.

Запрос - просто забирает первый элемент из левой части и снова восстанавливает баланс.

Сложность:

Время: O(N)

Память: O(N)

Код:

```
/**
 * @author: Slava Yakimenko
 */


#include <deque>
#include <iostream>


using namespace std;
typedef long long ll;


#define fastIO()                  \
  do {                            \
    ios::sync_with_stdio(false); \
    cin.tie(nullptr);             \
    cout.tie(nullptr);            \
  } while (0)
#define deb(x) cout << #x << " = " << x << '\n'
#define all(x) x.begin(), x.end()
```

```
#define el '\n'
#define INF static_cast<ll>(1e17)

/*
 * HEADER END
 */

void balance(deque<ll>& left, deque<ll>& right) {
  if (left.size() < right.size()) {
    left.push_back(right.front());
    right.pop_front();
  }
  if (left.size() > right.size() + 1) {
    right.push_front(left.back());
    left.pop_back();
  }
}

void solve() {
  ll n = 0;
  cin >> n;

  deque<ll> left;
  deque<ll> right;

  for (ll i = 0; i < n; ++i) {
    char type = 0;
    cin >> type;

    if (type == '+') {
      ll x = 0;
      cin >> x;
      right.push_back(x);
      balance(left, right);
    } else if (type == '*') {
      ll x = 0;
      cin >> x;
```

```
      right.push_front(x);
      balance(left, right);
    } else {
      cout << left.front() << el;
      left.pop_front();
      balance(left, right);
    }
  }
}

int main() {
  fastIO();
  // freopen("input.txt", "r", stdin);
  // freopen("output.txt", "w", stdout);
  bool multitest = false;
  ll T = 1;
  if (multitest) {
    cin >> T;
  }
  for (ll i = 0; i < T; ++i) {
    solve();
  }
}
```

Задача «K. Менеджер памяти-1»

Есть массив из N ячеек памяти и M запросов. Положительный запрос означает выделение блока памяти длины K, отрицательный — освобождение памяти, выделенной в запросе с номером T. Нужно эмулировать работу менеджера памяти.

Формат ввода

В первой строке задаются N и M. Далее идут M запросов: число K для выделения или число -T для освобождения памяти, связанной с запросом T.

Формат вывода

Для каждого запроса на выделение нужно вывести адрес первой ячейки выделенного блока, либо -1, если подходящего блока нет.

Идея решения:

Свободные и занятые участки памяти я храню как двусвязный список отрезков. Это позволяет быстро склеивать соседние свободные куски после освобождения памяти.

Все свободные отрезки дополнительно кладу в приоритетную очередь. В вершине лежит самый выгодный свободный блок: сначала максимальная длина, а при равенстве — самый левый.

Из-за разбиений и склеек в куче появляются устаревшие записи, поэтому перед использованием вершины я проверяю, что она все еще соответствует живому свободному отрезку.

При выделении либо беру весь сегмент, либо делю его на занятый левый кусок и оставшийся свободный правый. При освобождении склеиваю текущий сегмент с соседями, если они свободны.

Сложность:

Время: O(M log M)

Память: O(M)

Код:

```
/**
 * @author: Slava Yakimenko
 */

#include <iostream>
#include <queue>
#include <utility>
#include <vector>

using namespace std;
typedef long long ll;

#define fastIO()                 \
  do {                           \
    ios::sync_with_stdio(false); \
    cin.tie(nullptr);            \
    cout.tie(nullptr);           \
  } while (0)
#define deb(x) cout << #x << " = " << x << '\n'
#define all(x) x.begin(), x.end()
#define el '\n'
#define INF static_cast<ll>(1e17)

/*
 * HEADER END
 */
```

```
struct Node {
  ll l = 0;
  ll r = 0;
  int prev = 0;
  int next = 0;
  bool is_free = false;
  bool alive = false;
};

struct HeapNode {
  ll len = 0;
  ll neg_l = 0;
  int id = 0;

  bool operator<(const HeapNode& other) const {
    if (len != other.len) {
      return len < other.len;
    }
    if (neg_l != other.neg_l) {
      return neg_l < other.neg_l;
    }
    return id < other.id;
  }
};

ll get_len(const Node& node) {
  return node.r - node.l + 1;
}

void push_free_segment(priority_queue<HeapNode>& heap, const vector<Node>& nodes, int id) {
  heap.push({get_len(nodes[id]), -nodes[id].l, id});
}

bool is_actual_free_segment(const HeapNode& top, const vector<Node>& nodes) {
  const Node& node = nodes[top.id];
  return node.alive && node.is_free && get_len(node) == top.len && node.l == -top.neg_l;
}
```

```
void solve() {
  ll n = 0, m = 0;
  cin >> n >> m;

  vector<Node> nodes(m + 2);
  vector<int> allocated(m + 1, 0);
  priority_queue<HeapNode> heap;

  nodes[1] = {1, n, 0, 0, true, true};
  int nodes_cnt = 1;
  push_free_segment(heap, nodes, 1);

  for (ll i = 1; i <= m; ++i) {
    ll x = 0;
    cin >> x;

    if (x > 0) {
      while (!heap.empty() && !is_actual_free_segment(heap.top(), nodes)) {
        heap.pop();
      }

      if (heap.empty() || heap.top().len < x) {
        cout << -1 << el;
        continue;
      }

      const int id = heap.top().id;
      heap.pop();

      allocated[i] = id;
      cout << nodes[id].l << el;

      if (get_len(nodes[id]) == x) {
        nodes[id].is_free = false;
        continue;
      }
```

```
      ++nodes_cnt;
      nodes[nodes_cnt] = {nodes[id].l + x, nodes[id].r, id, nodes[id].next, true, true};
      if (nodes[id].next != 0) {
        nodes[nodes[id].next].prev = nodes_cnt;
      }

      nodes[id].r = nodes[id].l + x - 1;
      nodes[id].next = nodes_cnt;
      nodes[id].is_free = false;

      push_free_segment(heap, nodes, nodes_cnt);
    } else {
      const ll t = -x;
      if (t < 1 || t > m || allocated[t] == 0) {
        continue;
      }

      int id = allocated[t];
      allocated[t] = 0;
      if (!nodes[id].alive || nodes[id].is_free) {
        continue;
      }

      nodes[id].is_free = true;
      int cur = id;

      if (nodes[cur].prev != 0 && nodes[nodes[cur].prev].alive &&
nodes[nodes[cur].prev].is_free) {
        const int prev_id = nodes[cur].prev;
        nodes[prev_id].r = nodes[cur].r;
        nodes[prev_id].next = nodes[cur].next;
        if (nodes[cur].next != 0) {
          nodes[nodes[cur].next].prev = prev_id;
        }
        nodes[cur].alive = false;
        cur = prev_id;
      }
```

```
      if (nodes[cur].next != 0 && nodes[nodes[cur].next].alive &&
nodes[nodes[cur].next].is_free) {
        const int next_id = nodes[cur].next;
        nodes[cur].r = nodes[next_id].r;
        nodes[cur].next = nodes[next_id].next;
        if (nodes[next_id].next != 0) {
          nodes[nodes[next_id].next].prev = cur;
        }
        nodes[next_id].alive = false;
      }

      nodes[cur].is_free = true;
      push_free_segment(heap, nodes, cur);
    }
  }
}

int main() {
  fastIO();
  // freopen("input.txt", "r", stdin);
  // freopen("output.txt", "w", stdout);
  bool multitest = false;
  ll T = 1;
  if (multitest) {
    cin >> T;
  }
  for (ll i = 0; i < T; ++i) {
    solve();
  }
}
```

Задача «L. Минимум на отрезке»

По массиву длины N двигается окно длины K. Для каждого положения окна нужно найти минимум среди элементов, которые в него входят.

Формат ввода

В первой строке задаются N и K. Во второй строке записаны N чисел массива.

Формат вывода

Нужно вывести минимумы для всех положений окна.

Идея решения:

Для решения удобно поддерживать монотонную очередь индексов.

Перед добавлением нового элемента я убираю из начала дека индексы, которые уже вышли из окна.

Потом убираю из конца все индексы, у которых значение не меньше текущего, потому что они уже никогда не смогут стать минимумом.

После этого минимум для текущего окна всегда лежит в начале дека.

Сложность:

Время: O(N)

Память: O(K)

Код:

```
/**
 * @author: Slava Yakimenko
 */


#include <deque>
#include <iostream>
#include <vector>


using namespace std;
typedef long long ll;


#define fastIO()                 \
  do {                           \
    ios::sync_with_stdio(false); \
    cin.tie(nullptr);            \
    cout.tie(nullptr);           \
  } while (0)
#define deb(x) cout << #x << " = " << x << '\n'
#define all(x) x.begin(), x.end()
#define el '\n'
#define INF static_cast<ll>(1e17)


/*
 * HEADER END
```

```
 */

void solve() {
  ll n = 0, k = 0;
  cin >> n >> k;

  vector<ll> a(n);
  for (ll i = 0; i < n; ++i) {
    cin >> a[i];
  }

  deque<ll> q;
  bool first = true;
  for (ll i = 0; i < n; ++i) {
    while (!q.empty() && q.front() <= i - k) {
      q.pop_front();
    }

    while (!q.empty() && a[q.back()] >= a[i]) {
      q.pop_back();
    }
    q.push_back(i);

    if (i + 1 >= k) {
      if (!first) {
        cout << ' ';
      }
      first = false;
      cout << a[q.front()];
    }
  }
  cout << el;
}

int main() {
  fastIO();
  // freopen("input.txt", "r", stdin);
```

```
  // freopen("output.txt", "w", stdout);
  bool multitest = false;
  ll T = 1;
  if (multitest) {
    cin >> T;
  }
  for (ll i = 0; i < T; ++i) {
    solve();
  }
}
```